A B C A A B C A A A k=2

valid window → make all chars the most frequent one

window length - max freq ≤ k ⟺ valid window

A B C A A B C A A A res = 7

- if valid window - expand one step to the right
- if not - slide the window to the right

- Counts of each char in the window (at most 26 entries)
  - For a window, go through counts map / list & figure out max freq
  - O(26n)

- {A: 3, B: 4}
  {
  4: {A, B}
  }

- Maintain max freq in Θ(1) time by maintaining:
  1. char to counts map
  2. counts to chars map

A A A A B C C C C C C k=1

A: 0, B: 1, C: 6

max-freq = 6

MaxPQ

- insert(val)
- removeMax()
- getMax()
- size()
- isEmpty()

IndexMaxPQ

- insert(index, val)
- change(index, val)
- contains(index) — $\Theta(1)$
- remove(index)
- getMax() — $\Theta(1)$
- getMaxIndex() — $\Theta(1)$
- removeMax()
- size() — $\Theta(1)$
- isEmpty() — $\Theta(1)$

0	1	2	3	4	5	6	7
		2	3%3 =0	4%3 =1			

index in pq = ind % k

remainders when divided k : $[0, k-1]$ — k remainder possible

pq = $[i_2, i_1, i_3]$ (0, 1, 2)

qp = $[1, 2, 0]$ (i_1, i_2, i_3)

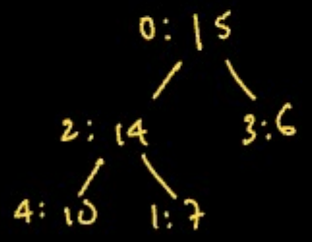
values = $[V_1, V_2, V_3]$

array forming a complete binary tree

0 - i_1 - V_1

1 - i_2 - V_2

2 - i_3 - V_3



	0	(1)	2	3	4
pq	2	0	3	4	1
qp	1	4	0	2	3
values	15	7	14	6	10

→ n elements 0 to n-1

Monotonic queue

0	1	2	3	4	5	6	7	8	9	
4	2	100	100	20	19	13	14	13	13	25
100	100	100	100	20	17	14	14	13	13	

(100, 3), (20, 4), (19, 5), (14, 7), (13, 9)

	0	1	2	3	4	5	6	7
	1	3	-1	-3	5	3	6	7
prefix max	1	3	3	-3	5	5	6	7
suffix max	3	3	-1	5	5	3	7	7

max(i, j) = max(suffix[i], prefix[j])

[A D O B E C O D E B A] N C A

{A: 2
B: 2
C: 1}

min = 11

A B C A uc = 3

{A: 2
B: 1
C: 1}

count = 3

↓

#chars for which count is met (≤ 0 in the map)